

Guía de QA

FibexSign — 59 casos de prueba manuales

GrupoConex — Plataforma de firma electrónica self-hosted ("Firma")

Generado: 2026-09-24

Hecho por: Ciclo Fibex (GDA)

Casos de prueba manuales derivados de los 14 documentos técnicos de `docs/`,
verificados contra el código real. Complementa (no reemplaza) el testing
automatizado.

Última actualización: 2026-09-24 **Alcance:** casos de prueba manuales derivados de los 14 documentos técnicos de `docs/` (verificados contra el código real). No reemplaza al testing automatizado (`docs/testing-guide.md`), lo complementa desde la perspectiva de un tester manual/exploratorio.

Cómo usar esta guía

Cada caso de prueba sigue el formato: Precondición, Pasos, Resultado esperado, Prioridad y una Nota opcional cuando el caso cubre un edge case real del código (no algo que un tester probaría por intuición). Los casos marcados como "regresión" están anclados a un comportamiento específico ya verificado en el código — si ese comportamiento cambia sin que el caso siga pasando, es una señal real de regresión, no solo un cambio de expectativas.

Entorno de pruebas

Ver `docs/deployment-guide.md` para levantar el stack completo (`docker-compose up`) y `docs/environment-variables.md` para las variables mínimas requeridas (Mongo, storage S3/local, Mailgun/SMTP, certificado PFX/P12 de firma). La mayoría de los casos de esta guía asumen: servidor backend accesible, MongoDB accesible, al menos un usuario admin de prueba ya creado, y un servicio de email configurado (real o capturado, p.ej. Mailhog) para verificar los correos de OTP/notificación.

Índice

1. Acceso, autenticación, multitenancy y rate limiting
2. Preparación, envío y firma de documentos
3. Email Builder, OpenSignDrive, Reportes/Contactos e Internacionalización
4. Casos de seguridad conocidos (sin corregir)

QA Guide: Acceso, Autenticación, Multitenancy y Rate Limiting

Última actualización: 2026-09-24

Fuentes verificadas contra:

- `docs/authentication-and-multitenancy.md`
 - `docs/rate-limiting.md`
 - `apps/OpenSignServer/cloud/parsefunction/SendMailOTPv1.js` (OTP_EXPIRY_MS, OTP generation)
 - `apps/OpenSignServer/cloud/parsefunction/AuthLoginAsMail.js` (MAX_OTP_ATTEMPTS)
 - `apps/OpenSignServer/index.js` (account lockout defaults, TRUST_PROXY_HOPS)
 - `apps/OpenSign/src/pages/Login.jsx` , `GuestLogin.jsx`
-

1. Login con Contraseña

TC-AUTH-01 — Login exitoso con credenciales válidas

- **Precondición:** Usuario registrado con email y contraseña. Servidor accesible.
- **Pasos:**
 1. Navegar a la pantalla de login (p.ej. `/login`).
 2. Ingresar email y contraseña válidos.
 3. Hacer clic en el botón de envío de formulario.
- **Resultado esperado:**
 - El servidor responde con `Parse.User.logIn()` exitoso.
 - Se devuelve `sessionToken` al cliente.
 - El cliente almacena `accesstoken` , `UserInfo` y la clave Parse en `localStorage` .
 - Se resuelve `contracts_Users` por el `_User` recién autenticado.
 - El navegador redirige a la página siguiente (e.g., dashboard) según el rol del usuario.
- **Prioridad:** Alta

TC-AUTH-02 — Login fallido con credenciales inválidas

- **Precondición:** Usuario registrado. Servidor accesible.
- **Pasos:**
 1. Navegar a `/login`.
 2. Ingresar email válido pero contraseña incorrecta.
 3. Hacer clic en envío.
- **Resultado esperado:**
 - Parse Server retorna `OBJECT_NOT_FOUND` o `Parse.Error` de validación de credenciales.
 - El cliente interpreta el error y muestra un mensaje localizado: `"invalid-username-password-region"`.
 - El usuario permanece en la pantalla de login.
 - El contador de intentos fallidos de la cuenta se incrementa (verificar en backend: Parse Server `accountLockout`).
- **Prioridad:** Alta

TC-AUTH-03 — Bloqueo de cuenta tras 5 intentos fallidos

- **Precondición:**
 - Usuario registrado (email + contraseña).
 - Servidor configurado con `ACCOUNT_LOCKOUT_THRESHOLD=5` y `ACCOUNT_LOCKOUT_DURATION_MINUTES=5` (defaults).
 - **Pasos:**
 1. Navegar a `/login`.
 2. Ingresar email válido.
 3. Intentar login con contraseña incorrecta **5 veces consecutivas**.
 4. En el 6º intento, ingresar contraseña **correcta**.
 - **Resultado esperado:**
 - Tras el 5º intento fallido, Parse Server bloquea la cuenta automáticamente.
 - En el 6º intento, incluso con contraseña correcta, se rechaza con un mensaje de cuenta bloqueada o error equivalente.
 - El cliente debe mostrar un mensaje como `"too-many-login-attempts"` o equivalente localizado.
 - Tras esperar 5 minutos (duración de bloqueo por defecto), la cuenta se desbloquea y se permite login nuevamente.
 - **Prioridad:** Alta
 - **Nota:** Verificar en `apps/OpenSignServer/index.js` líneas 111-112 que los defaults están configurados. Si están diferentes en la instalación, ajustar los valores de paso y duración de espera en consecuencia.
-

2. Login sin Contraseña (OTP) — Firmante Invitado

TC-AUTH-04 — Solicitud de código OTP para contacto conocido

- **Precondición:**
 - Documento creado y asignado a un firmante.
 - El email del firmante ya existe en `contracts_Contactbook` del tenant (contacto conocido, vinculado previamente).
 - Sistema de email funcionando (SMTP o Mailgun).
- **Pasos:**
 1. Abrir enlace de firma (típicamente `/load/recipientSignPdf/:docId/:contactId` con email precompletado o navegación desde el documento).
 2. Si se pide, ingresar el email del firmante en la pantalla de `GuestLogin`.
 3. Hacer clic en el botón "Solicitar código OTP" (o equivalente en la UI).
- **Resultado esperado:**
 - Se ejecuta `Parse.Cloud.run('SendOTPMailV1', { email, docId })`.
 - El servidor:
 - Genera un código numérico de **4 dígitos** (rango 1000–9999).
 - Envía el OTP por email al contacto.
 - Almacena el OTP en `defaultdata_Otp` con `Email`, `OTP`, `ExpiresAt` (ahora + 10 minutos), `FailedAttempts: 0`.
 - El cliente recibe confirmación y muestra campo de entrada para el OTP.
 - El correo llega en pocos segundos (dependiendo del proveedor).
- **Prioridad:** Alta

TC-AUTH-05 — Solicitud de OTP para contacto nuevo (crear contacto sobre la marcha)

- **Precondición:**

- Documento creado y lista de firmantes incluye un email que **no existe** en `contracts_Contactbook` del tenant (contacto nuevo).
- Usuario invitado no tiene cuenta en la plataforma.

- **Pasos:**

1. Abrir enlace de firma con parámetro de email desconocido (e.g., `?email=new-signer@example.com`).
2. El cliente detecta que el contacto no existe en la base de datos y muestra formulario opcional ("Completar datos del contacto").
3. Ingresar el email (ya prefillado), nombre, teléfono, puesto, empresa (opcional).
4. Hacer clic en botón para solicitar OTP.

- **Resultado esperado:**

- Se ejecuta `Parse.Cloud.run('linkcontacttodoc', { email, docId, ... })` (si el formulario se completa).
- El servidor crea un nuevo registro en `contracts_Contactbook` con `UserRole: 'contracts_Guest'`.
- Luego continúa con `SendOTPMailV1` (igual a TC-AUTH-04).
- El OTP se envía por email al contacto nuevo.
- El cliente avanza a la pantalla de entrada del código OTP.

- **Prioridad:** Alta

TC-AUTH-06 — Verificación de OTP correcta

- **Precondición:**
 - OTP solicitado en TC-AUTH-04 o TC-AUTH-05.
 - Código OTP válido en `defaultdata_Otp`, no expirado, `FailedAttempts < 5`.
- **Pasos:**
 1. Usuario recibe correo con el OTP (e.g., "Your OTP is **1234**").
 2. En la pantalla de `GuestLogin`, ingresar el código exacto (e.g., **1234**).
 3. Hacer clic en botón de verificación.
- **Resultado esperado:**
 - Se ejecuta `POST /functions/AuthLoginAsMail { email, otp }`.
 - El servidor:
 - Consulta `defaultdata_Otp` por `Email`.
 - Valida: `!isExpired && !isLockedOut && resOtp === otp`.
 - Resuelve el `Parse.User` con ese email.
 - Llama internamente `POST /loginAs` con Master Key → devuelve `sessionToken` legítimo.
 - Si `emailVerified: false`, marca en `true`.
 - El cliente recibe `{ sessionToken, ...user }`.
 - Ejecuta `Parse.User.become(sessionToken)`.
 - Almacena `accesstoken`, `UserInfo`, clave `Parse` en `localStorage`.
 - Navega a `/load/recipientSignPdf/:docId/:contactId`.
 - Desde ese punto, el firmante invitado tiene una sesión `Parse` completa.
- **Prioridad:** Alta

TC-AUTH-07 — Verificación de OTP incorrecto — incremento de intentos fallidos

- **Precondición:**
 - OTP válido en `defaultdata_Otp`.
 - Usuario intenta ingresar un código incorrecto (e.g., **9999** en lugar de **1234**).
- **Pasos:**
 1. En pantalla de entrada del OTP, ingresar código incorrecto.
 2. Hacer clic en verificación.
- **Resultado esperado:**
 - El servidor consulta `defaultdata_Otp` por `Email` y valida el OTP.
 - Detecta que no coincide.
 - Retorna mensaje: "Invalid Otp" (constante `GENERIC_INVALID_OTP_MESSAGE`).
 - Incrementa `FailedAttempts` en 1 (línea 94 de `AuthLoginAsMail.js`).
 - El cliente muestra mensaje de error y permite reintentar.
 - La sesión NO se establece (no hay redirección).
- **Prioridad:** Alta

TC-AUTH-08 — Bloqueo de OTP tras 5 intentos fallidos

- **Precondición:**
 - OTP válido en `defaultdata_Otp` con `FailedAttempts: 0`.
 - `MAX_OTP_ATTEMPTS = 5` (constante en `AuthLoginAsMail.js` línea 4).
- **Pasos:**
 1. Solicitar OTP (TC-AUTH-04).
 2. Intentar verificar el OTP **5 veces con código incorrecto**.
 3. En el 6º intento, ingresar el **código correcto**.
- **Resultado esperado:**
 - Tras el 5º intento fallido, `FailedAttempts` alcanza 5.
 - En el 6º intento (incluso con código correcto):
 - El servidor valida `isLockedOut = failedAttempts >= MAX_OTP_ATTEMPTS` → `true`.
 - Rechaza la verificación con `"Invalid Otp"`.
 - **No incrementa** `FailedAttempts` (línea 93–96: el `set()` de `FailedAttempts` se omite si ya está bloqueado).
 - El usuario debe solicitar un nuevo OTP (empezar de nuevo con TC-AUTH-04).
- **Prioridad:** Alta
- **Nota:** A diferencia del bloqueo de cuenta de login con contraseña (que dura 5 minutos), el bloqueo de OTP requiere solicitar un nuevo código, no hay desbloqueo automático por tiempo. Verificar en `SendMailOTPV1.js` línea 66 que se resetea `FailedAttempts: 0` al generar un nuevo OTP.

TC-AUTH-09 — Expiración del código OTP

- **Precondición:**
 - OTP solicitado hace **más de 10 minutos**.
 - `OTP_EXPIRY_MS = 10 * 60 * 1000` (línea 4 de `SendMailOTPV1.js`).
- **Pasos:**
 1. Solicitar OTP (recibe código por email).
 2. Esperar **10 minutos y 1 segundo**.
 3. Ingresar el código original y hacer clic en verificación.
- **Resultado esperado:**
 - El servidor consulta `defaultdata_Otp` y valida el timestamp `ExpiresAt`.
 - Detecta `isExpired = expiresAt < Date.now()` → `true`.
 - Rechaza con `"Invalid Otp"`.
 - **No incrementa** `FailedAttempts` (línea 93–96: omitido si expirado).
 - El usuario debe solicitar un nuevo OTP.
- **Prioridad:** Media
- **Nota:** Tiempo de expiración exacto es 10 minutos desde la solicitud, no desde la recepción del email. Considerar tiempo de envío del email al validar en pruebas manuales (normalmente < 5 segundos).

3. Sesión y Ciclo de Vida

TC-AUTH-10 — Sesión establecida con `Parse.User.become(sessionToken)`

- **Precondición:**
 - Login completado (con contraseña o OTP).
 - Cliente ha ejecutado `Parse.User.become(sessionToken)`.
 - `sessionToken` almacenado en `localStorage.accesstoken`.
- **Pasos:**
 1. Realizar una acción autenticada (e.g., ver perfil, listar documentos, crear documento).
 2. Inspeccionar un request outgoing en Network inspector → header `X-Parse-Session-Token`.
- **Resultado esperado:**
 - El cliente incluye `X-Parse-Session-Token: <sessionToken>` en cada Cloud Function.
 - Parse Server resuelve la sesión y puebla `request.user` (línea 32–42 de `authentication-and-multitenancy.md`).
 - Las Cloud Functions validan `if (!request.user) throw Parse.Error(...)` y prosiguen.
 - Operaciones se ejecutan correctamente.
- **Prioridad:** Alta

TC-AUTH-11 — Sesión expirada — visualización de `SessionExpiredModal`

- **Precondición:**
 - Usuario autenticado con sesión activa.
 - `localStorage.TenantId` y `Parse.User.current().getSessionToken()` existen.
 - Sesión ha expirado (manualmente borrada, o timeout del servidor alcanzado, o sesión revocada).
 - **Pasos:**
 1. Usuario intenta realizar una acción autenticada (e.g., hacer clic en "Ver documentos").
 2. `withSessionValidation` (HOF en `utils/withSessionValidation.js`) valida antes de ejecutar.
 - **Resultado esperado:**
 - `withSessionValidation` detecta que `sessionToken` es inválido o falta.
 - Despacha `sessionStatus(false)` a Redux.
 - `HomeLayout.jsx` o ruta `Validate.jsx` dejan de renderizar el `Outlet`.
 - Muestra modal `SessionExpiredModal`.
 - Usuario hace clic en botón de cierre/logout → `Parse.User.logout()` se ejecuta.
 - Redirige a `/login`.
 - **Prioridad:** Alta
 - **Nota:** `withSessionValidation` es un guard de **UX en el cliente**, no reemplaza autorización en servidor. No es un test de seguridad de backend.
-

4. Multitenancy

TC-AUTH-12 — Aislamiento de datos entre tenants

- **Precondición:**
 - Dos tenants distintos (p.ej., "Empresa A" y "Empresa B"), ambos con usuarios y documentos.
 - Usuario A logueado en Empresa A.
 - Empresa B tiene documentos y usuarios que no deben ser visibles a Usuario A.
- **Pasos:**
 1. Usuario A se loguea (credenciales de Empresa A).
 2. Se almacena `TenantId` de Empresa A en `localStorage.TenantId`.
 3. Acceder a endpoint para listar documentos (p.ej., `Parse.Cloud.run('filterDocs', { ... })`).
- **Resultado esperado:**
 - La Cloud Function resuelve `request.user` → busca `contracts_Users` → obtiene `TenantId` del usuario.
 - Filtra documentos por ese `TenantId` (verificar en el código que la query incluya `equalTo('TenantId', ...)` o equivalente).
 - Solo documentos de Empresa A son devueltos.
 - Documentos de Empresa B no aparecen, incluso si existen en la base de datos.
- **Prioridad:** Alta
- **Nota:** El aislamiento es **lógico** (filtro SQL/Mongo), no físico (no hay bases de datos separadas). Verificar en `filterDocs.js` u otra función que realice queries que incluyan el filtro de `TenantId`. Si no lo hace, es un defecto de seguridad.

TC-AUTH-13 — Validación de TenantId en queries multiorganizacionales

- **Precondición:**
 - Usuario Admin de Empresa A, con acceso a múltiples organizaciones dentro de Empresa A.
 - Endpoint que requiere `organizationId` (e.g., `getUserListByOrg({ organizationId })`).
 - Organización X pertenece a Empresa A, Organización Y pertenece a Empresa B.
 - **Pasos:**
 1. Usuario Admin de Empresa A se loguea.
 2. Llamar a `Parse.Cloud.run('getUserListByOrg', { organizationId: 'Y' })` (intentar acceder a org de Empresa B).
 - **Resultado esperado:**
 - La Cloud Function:
 - Valida `request.user` (debe existir).
 - Resuelve `contracts_Users` del llamante → obtiene su `TenantId` (de Empresa A).
 - Resuelve `contracts_Organizations` con ID Y → obtiene su `TenantId` (de Empresa B).
 - Compara: `targetOrg.TenantId !== callerTenantId` → mismatch.
 - Lanza `Parse.Error(OPERATION_FORBIDDEN, 'Unauthorized.')`.
 - El cliente recibe error 403.
 - **No se devuelven** usuarios de la Organización Y.
 - **Prioridad:** Alta
 - **Nota:** Verificar en `getUserListByOrg.js` líneas 36–39 que el chequeo de `TenantId` ocurra y lance error. Este es el mecanismo principal de aislamiento multi-organización.
-

5. Rate Limiting: Bucket Estricto (Autenticación)

TC-AUTH-14 — Exceder límite en bucket estricto de autenticación

- **Precondición:**

- `AUTH_RATE_LIMIT_WINDOW_MS = 300000` (default: 5 minutos).
- `AUTH_RATE_LIMIT_MAX = 30` (default: 30 requests en 5 minutos por IP+ruta).
- Cliente tiene una IP única (o simula una desde un entorno controlado).

- **Pasos:**

1. Realizar **30 requests exitosos** a una ruta estricta de autenticación dentro de 5 minutos. Ejemplos de rutas estrictas (sufijos):

- `/login`
- `/functions/loginuser`
- `/functions/AuthLoginAsMail`
- `/functions/SendOTPMailV1`

2. Realizar un **31º request** a la misma ruta dentro de la ventana.

- **Resultado esperado:**

- Los primeros 30 requests se procesan normalmente.
- El 31º request retorna:
 - **Código HTTP:** `429 Too Many Requests`
 - **Body:** `{ error: 'Too many requests, please try again later.' }`
 - **Headers:** `RateLimit-*` estándar (según `express-rate-limit` con `standardHeaders: true`).
- El cliente recibe 429 y no puede continuar (debe esperar a que expire la ventana de 5 minutos).

- **Prioridad:** Alta

- **Nota:** El límite se cuenta por **IP + ruta exacta**. Un request a `/functions/loginuser` y otro a `/functions/SendOTPMailV1` se cuentan en buckets separados (ambos dentro del store estricto).

TC-AUTH-15 — Exceder límite en bucket operacional (navegación)

- **Precondición:**
 - Ruta operacional (sufijo):
 - `/functions/getUserDetails`
 - `/functions/declinedoc`
 - `OPERATIONAL_RATE_LIMIT_MAX = 300` (default: 300 requests en 5 minutos por IP+ruta).
 - Cliente autenticado.
- **Pasos:**
 1. Realizar **300 requests exitosos** a una ruta operacional dentro de 5 minutos.
 2. Realizar un **301º request** a la misma ruta.
- **Resultado esperado:**
 - Los primeros 300 requests se procesan.
 - El 301º retorna `429 Too Many Requests` con el mismo cuerpo y headers.
 - El cliente no puede continuar.
- **Prioridad:** Media
- **Nota:** El bucket operacional es **10 veces más permisivo** que el estricto (300 vs 30). Verificar en `authRateLimiter.js` línea 12 que el default sea `AUTH_RATE_LIMIT_MAX * 10` si `OPERATIONAL_RATE_LIMIT_MAX` no está seteado.

TC-AUTH-16 — Rate limit por IP+ruta: aislamiento entre rutas

- **Precondición:**
 - Cliente con IP fija.
 - Dos rutas estrictas distintas: `/functions/loginuser` y `/functions/SendOTPMailV1`.
 - **Pasos:**
 1. Realizar **30 requests** a `/functions/loginuser`.
 2. Realizar un **31º request** a `/functions/loginuser` → esperado: 429.
 3. Inmediatamente, realizar un **1º request** a `/functions/SendOTPMailV1` (diferente ruta).
 - **Resultado esperado:**
 - El 31º request a `/functions/loginuser` retorna 429 (como en TC-AUTH-14).
 - El 1º request a `/functions/SendOTPMailV1` se procesa **exitosamente** (200 OK o respuesta normal).
 - Los contadores son **independientes por ruta** (clave: `IP:ruta`).
 - **Prioridad:** Media
 - **Nota:** Verificar en `authRateLimiter.js` línea 49 que la clave sea `${ipKeyGenerator(request.ip)}:${request.path}` (no solo IP).
-

6. Rate Limiting: Configuración de Proxy

TC-AUTH-17 — Rate limit aísla atacantes cuando TRUST_PROXY_HOPS = 0 (sin proxy)

- **Precondición:**
 - `TRUST_PROXY_HOPS = 0` (default, o no seteado).
 - Servidor **sin proxy/load balancer** entre cliente e instancia de Parse Server (conexión directa).
 - Dos IPs de cliente distintas (e.g., 203.0.113.1 y 203.0.113.2).
- **Pasos:**
 1. Cliente A (IP: 203.0.113.1) realiza **30 requests** a `/functions/loginuser` en 5 minutos.
 2. Cliente A realiza un **31º request** → esperado: 429.
 3. Cliente B (IP: 203.0.113.2) realiza un **1º request** a `/functions/loginuser` en la misma ventana.
- **Resultado esperado:**
 - Cliente A recibe 429 en el 31º request.
 - Cliente B recibe 200 OK (o respuesta normal) en el 1º request.
 - El contador de Cliente A (**203.0.113.1:loginuser**) es independiente del de Cliente B (**203.0.113.2:loginuser**).
 - Esto demuestra que el rate limiting **aísla atacantes de usuarios legítimos** por IP.
- **Prioridad:** Alta
- **Nota:** Este caso valida que `request.ip` resuelve correctamente cuando no hay proxy. Verificar en `index.js` línea 186-192 que `app.set('trust proxy', false)` cuando `TRUST_PROXY_HOPS = 0`.

TC-AUTH-18 — Rate limit detrás de proxy con TRUST_PROXY_HOPS incorrecto

- **Precondición:**
 - Servidor **detrás de un proxy/load balancer** (p.ej., Nginx, CloudFlare).
 - `TRUST_PROXY_HOPS = 0` (default, pero **incorrecto** para esta topología).
 - Dos usuarios legítimos distintos accediendo a través del proxy (misma IP de proxy).
- **Pasos:**
 1. Usuario A (IP real: 203.0.113.1, pero llega vía proxy con IP: 192.168.1.100) realiza **30 requests** a `/functions/loginuser`.
 2. Usuario B (IP real: 203.0.113.2, también llega vía proxy IP: 192.168.1.100) realiza un **1º request** a `/functions/loginuser**`.
- **Resultado esperado (incorrecto, demostrando el fallo):**
 - Sin `TRUST_PROXY_HOPS` seteado correctamente, Express usa `request.connection.remoteAddress` (IP del proxy: 192.168.1.100).
 - El rate limiter ve ambos requests como proviniendo de la misma IP (192.168.1.100).
 - Contador compartido: **192.168.1.100:loginuser** = 31 requests.
 - Usuario B recibe **429** (bloqueado incorrectamente por el ataque de Usuario A).
 - **Esto es un fallo:** usuarios legítimos se afectan mutuamente.
- **Prioridad:** Alta
- **Nota:** Este caso evidencia la importancia de `TRUST_PROXY_HOPS`. Servidor debe estar configurado con el valor correcto (p.ej., `TRUST_PROXY_HOPS = 1` para un proxy, `TRUST_PROXY_HOPS = 2` para proxy + load balancer). Verificar en `index.js` línea 191 el warning del console que se emite si no está seteado.

7. Casos de Regresión / Seguridad Documentados

TC-AUTH-19 — Validación de sesión no descarta requests sin Master Key de IPs no autorizadas

- **Precondición:**
 - `MASTER_KEY_IPS` configurado en `.env` (default: `['127.0.0.1', ':::1']`).
 - Cliente intenta enviar `X-Parse-Master-Key` desde una IP no autorizada (p.ej., 203.0.113.1).
- **Pasos:**
 1. Desde IP externa (no en `MASTER_KEY_IPS`), enviar request con header `X-Parse-Master-Key: <master-key>`.
 2. Llamar a una operación que requiere Master Key (p.ej., crear usuario con `useMasterKey: true`).
- **Resultado esperado:**
 - Parse Server valida la IP origen (`request.ip`).
 - Rechaza la request porque la IP no está en la lista blanca `MASTER_KEY_IPS`.
 - Retorna 403 Forbidden.
 - **Nota:** Este es un control **distinto e independiente** del rate limiting. Rate limiting limita volumen; `MASTER_KEY_IPS` restringe origen. Ambos dependen de `request.ip` resuelto correctamente (de ahí la importancia de `TRUST_PROXY_HOPS`).
- **Prioridad:** Alta
- **Nota:** Verificar en `index.js` línea 106-110 que `MASTER_KEY_IPS` está configurado. Si `TRUST_PROXY_HOPS = 0` detrás de un proxy, `request.ip` será la IP del proxy, no la del cliente real → el control falla. Caso relacionado a TC-AUTH-18.

TC-AUTH-20 — Validación manual de sesión en cada Cloud Function

- **Precondición:**
 - Cloud Function que requiere sesión (p.ej., `getTenant`, `addUser`).
- **Pasos:**
 1. Llamar a la función sin incluir `X-Parse-Session-Token` en los headers (o sin iniciar sesión).
- **Resultado esperado:**
 - La función valida `if (!request.user)` al inicio.
 - Lanza `Parse.Error(INVALID_SESSION_TOKEN, 'Invalid session token.')`.
 - Retorna 403 Unauthorized.
- **Prioridad:** Media
- **Nota:** No hay middleware centralizado; cada función es responsable. Verificar en `getTenant.js`, `addUser.js`, `updateTenant.js` que el chequeo esté presente (línea 39-41 de `authentication-and-multitenancy.md`). Este es un riesgo de arquitectura: una función nueva que olvide el chequeo quedará abierta.

Resumen Ejecutivo de Cobertura

Área	Casos	IDs
Login con contraseña	3	TC-AUTH-01 a 03

Login OTP	6	TC-AUTH-04 a 09
Sesión y ciclo de vida	2	TC-AUTH-10 a 11
Multitenancy	2	TC-AUTH-12 a 13
Rate limiting — buckets	3	TC-AUTH-14 a 16
Rate limiting — proxy	2	TC-AUTH-17 a 18
Regresión / Seguridad	2	TC-AUTH-19 a 20
TOTAL	20	

Notas de Implementación para Testers Manuales

1. Herramientas recomendadas:

- Postman o Insomnia para requests REST directas a `/functions/AuthLoginAsMail`.
- Browser DevTools (Network tab) para inspeccionar headers `X-Parse-Session-Token`.
- `curl` desde línea de comandos para simular IPs o headers customizados.
- Clock mock o manual en el servidor para acelerar pruebas de expiración de OTP.

2. Limpieza entre ciclos de prueba:

- Borrar `localStorage` completamente entre tests de sesión.
- Resetear la base de datos `defaultdata_Otp` si es necesario (borrando registros viejos).
- Usar `resetAuthRateLimiterStoreForTesting()` si se ejecutan tests de rate limiting consecutivos.

3. Ambiente de prueba:

- Usar una instancia de QA con email mock (no enviar correos reales).
- Configurar `ACCOUNT_LOCKOUT_THRESHOLD` y `ACCOUNT_LOCKOUT_DURATION_MINUTES` con valores bajos para acelerar pruebas (ej: `threshold=2`, `duration=1`).
- Configurar `TRUST_PROXY_HOPS` correctamente según la topología de infra.

4. Registro de defectos:

- Si TC-AUTH-12 falla (un usuario ve datos de otro tenant), escalar como **CRITICAL**.
- Si TC-AUTH-13 falla (validación de `TenantId` no se ejecuta), escalar como **CRITICAL**.
- Si TC-AUTH-17 o TC-AUTH-18 fallan (rate limiting no aísla por IP), escalar como **HIGH**.
- Si TC-AUTH-19 o TC-AUTH-20 fallan (Master Key o sesión no validados), escalar como **CRITICAL**.

Guía de QA: Flujo Core de Documentos

Última actualización: 2026-09-24

Sección: Preparación, envío y firma de documentos

Cobertura: `document-preparation-and-sending.md` , `signing-ceremony.md` , `reports.md` (plantillas)

Precondiciones globales

- Usuario autenticado y logueado en FibexSign
 - Entorno de prueba configurado con Parse Server, servicio de email y cifrado PKCS#12
 - PDF de prueba disponible (mínimo 1 página, máximo según `maxFileSize` configurado)
 - Contactos de prueba disponibles o emails de prueba vigentes
 - Certificado PKCS#12 (`.pfx`) configurado para el tenant o variables `PFX_BASE64` / `PASS_PHRASE` del entorno
 - Redis disponible (para OTP y cachés)
-

Preparación

TC-DOC-01 — Subir PDF válido y renderizar

- **Precondición:** Usuario en la pantalla de carga/creación de documento, sin documento previo cargado.
 - **Pasos:**
 1. Hacer clic en el área de carga (o selector `<input type="file">`) de `RenderAllPdfPage.jsx`
 2. Seleccionar un archivo PDF válido (mínimo 1 página, máximo dentro de `maxFileSize` configurado)
 3. Esperar a que el navegador procese y renderice el PDF página a página
 4. Desplazarse entre páginas usando los controles del componente `RenderPdf.jsx`
 5. Realizar un pinch-to-zoom (en móvil/trackpad) o zoom con rueda del ratón
 - **Resultado esperado:**
 - El PDF se renderiza página a página sin errores
 - Cada página se muestra escalada correctamente a la pantalla
 - Los controles de navegación permiten saltar entre páginas
 - El zoom (pinch y rueda) funciona sin distorsión
 - No hay alertas de error ni mensajes de rechazo
 - **Prioridad:** Alta
 - **Nota:** Validado en `RenderAllPdfPage.jsx:handleFileUpload` , línea ~45 (validación de tipo `application/pdf` y tamaño).
-

TC-DOC-02 — Rechazar archivo no-PDF

- **Precondición:** Usuario en la pantalla de carga de documento.
 - **Pasos:**
 1. Hacer clic en el área de carga
 2. Seleccionar un archivo de tipo no-PDF (por ejemplo, `.docx`, `.txt`, `.jpg`, `.xlsx`)
 3. Intentar cargar el archivo
 - **Resultado esperado:**
 - El archivo es rechazado
 - Se muestra un mensaje de error indicando que solo se aceptan archivos PDF
 - El documento no se carga
 - **Prioridad:** Alta
 - **Nota:** Validación en `RenderAllPdfPage.jsx:handleFileUpload`, línea ~50 (check `file.type === 'application/pdf'`).
-

TC-DOC-03 — Verificar tipos de widget disponibles

- **Precondición:** Un PDF válido está renderizado en el editor.
 - **Pasos:**
 1. Abrir el panel de widgets (`WidgetList.jsx` o similar) en el lateral/barra de herramientas
 2. Verificar la lista de widgets arrastrables disponibles
 3. Contar y listar cada tipo de widget
 4. Intentar arrastrar al menos tres tipos diferentes sobre el PDF
 - **Resultado esperado:**
 - Se encuentran disponibles estos 15 tipos de widget:
 - `signature`, `stamp`, `initials`, `textInputWidget`, `name`, `job title`, `company`, `email`, `date`, `textWidget`, `cellsWidget`, `checkbox`, `dropdown`, `radioButtonWidget`, `image`, `drawWidget`
 - Cada widget es arrastra- y soltable sobre cualquier página del PDF
 - Al soltar, el widget se posiciona correctamente según `containerScale` y tamaño por defecto (`defaultWidthHeight`)
 - **Prioridad:** Media
 - **Nota:** Catálogo verificado en `WidgetComponent.jsx:draggableItems` y `getWidgetType.jsx`. Mobile usa variante compacta (`isMobileView`).
-

TC-DOC-04 — Asignar un único destinatario

- **Precondición:** Un PDF con widgets está preparado en el editor.
 - **Pasos:**
 1. Abrir el selector de destinatarios (modal/panel de `SignersInput.jsx` en `EditTemplate.jsx`)
 2. Hacer clic en el campo multi-select de firmantes
 3. Escribir el nombre o email de un contacto existente o nuevo
 4. Seleccionar o crear un único contacto
 5. Verificar que aparece en la lista `RecipientList.jsx` con su color asignado
 - **Resultado esperado:**
 - El contacto aparece en la lista de destinatarios
 - Se le asigna un color distintivo (`nameColor` , `darkenColor`)
 - El contador de widgets necesarios para este firmante muestra 0 (vacío hasta colocar widgets)
 - **Prioridad:** Alta
 - **Nota:** Búsqueda via Cloud Function `getsigners` → `apps/OpenSignServer/cloud/parsefunction/getSigners.js` , filtra por `CreatedBy` del usuario autenticado.
-

TC-DOC-05 — Asignar múltiples destinatarios

- **Precondición:** Un PDF con widgets está preparado.
 - **Pasos:**
 1. Abrir el selector de destinatarios
 2. Agregar tres contactos diferentes (uno por uno, seleccionando o creando cada uno)
 3. Verificar que todos aparecen en `RecipientList.jsx`
 4. Usar los handles de drag para reordenar manualmente los destinatarios
 5. Verificar que el orden visual se actualiza
 - **Resultado esperado:**
 - Todos los contactos aparecen en la lista
 - El reorden es posible mediante drag-and-drop (`onSortEnd` de `arrayMove`)
 - El orden guardado es el visual (el array `Signers / Placeholders`)
 - **Prioridad:** Alta
 - **Nota:** Reorden usa `arrayMove` en `SignersInput.jsx` . El orden se persiste en `EditTemplate.jsx:formData.SendinOrder` .
-

TC-DOC-06 — Orden de firma como sugerencia (`SendinOrder=true`, `SendInOrderStrict=false`)

- **Precondición:** Un documento con tres destinatarios en orden específico está preparado.
 - **Pasos:**
 1. En `EditTemplate.jsx`, activar el flag `SendinOrder` (checkbox/toggle)
 2. Dejar desactivado `SendInOrderStrict`
 3. Guardar/crear el documento
 4. Enviar a los tres destinatarios
 5. Verificar que en el backend se persiste `SendinOrder: true`, `SendInOrderStrict: false` en `contracts_Document`
 - **Resultado esperado:**
 - El documento se envía correctamente
 - El backend registra `SendinOrder = true`
 - El backend registra `SendInOrderStrict = false`
 - Los firmantes pueden notificarse en orden (sugerencia visual/email), pero técnicamente cualquiera puede firmar en cualquier momento
 - **Prioridad:** Media
 - **Nota:** Persistencia en `createDocumentFromApp.js`, `createBatchDocs.js`, `saveAsTemplate.js`. Validación en backend solo si ambos flags son true.
-

TC-DOC-07 — Orden de firma forzoso (`SendinOrder=true`, `SendInOrderStrict=true`)

- **Precondición:** Un documento con tres destinatarios (Firmante A, B, C en ese orden) está preparado.
 - **Pasos:**
 1. En `EditTemplate.jsx`, activar ambos flags: `SendinOrder = true` y `SendInOrderStrict = true`
 2. Guardar/crear el documento
 3. Enviar a A, B, C
 4. Verificar en `contracts_Document` que ambos flags están `true`
 - **Resultado esperado:**
 - El documento se persiste con `SendinOrder: true`, `SendInOrderStrict: true`
 - La validación en `PDF.js:signPdf` estará activa (verificado en paso siguiente, TC-DOC-10)
 - **Prioridad:** Alta
 - **Nota:** Migración verificada en `20260430000000-add_sendinorderstrict_field.cjs`. Flag aplicado en `PDF.js` líneas 54-64.
-

Envío

TC-DOC-08 — Envío individual a un destinatario

- **Precondición:** Un documento preparado con un destinatario, widgets colocados, metadatos completos.
 - **Pasos:**
 1. Hacer clic en el botón "Enviar" o "Send"
 2. Seleccionar el método de envío individual (no bulk)
 3. Verificar que se llama la Cloud Function `createdocumentfromapp`
 4. Esperar a que se cree `contracts_Document` y se envíe el email
 5. Revisar la bandeja de entrada del destinatario para el email con link de firma
 - **Resultado esperado:**
 - Se crea un único registro `contracts_Document`
 - El email llega con el link de firma (forma `login/<base64(docId/email)>` o `recipientSignPdf/:docId/:contactId`)
 - El documento contiene todos los metadatos (Name, URL, Signers[], Placeholders[], SendinOrder, SendInOrderStrict, IsEnableOTP, etc.)
 - El contador de documentos del usuario se decrementa en 1
 - **Prioridad:** Alta
 - **Nota:** Cloud Function `createdocumentfromapp` → `apps/OpenSignServer/cloud/parsefunction/createDocumentFromApp.js`. Hook `beforeSave` valida longitudes; `afterSave` recalcula ACL.
-

TC-DOC-09 — Envío masivo (bulk send / quick send)

- **Precondición:** Una plantilla o documento en borrador con destinatarios y widgets colocados.
 - **Pasos:**
 1. Hacer clic en "Enviar masivo", "Quick send" o "Bulk send"
 2. Verificar que se abre un modal/formulario con control de concurrencia
 3. Enviar a 5-10 destinatarios diferentes
 4. Esperar a que se creen los documentos (Cloud Function `batchdocuments`)
 5. Revisar que cada destinatario recibió un email individual
 6. Verificar que el dueño recibió un email resumen
 - **Resultado esperado:**
 - Se crean N documentos `contracts_Document` (uno por destinatario)
 - Cada destinatario recibe un email con su link de firma individual
 - El dueño recibe un email resumen vía `sendOwnerSummaryEmail`
 - Se usa `chunkArray` + `mapWithConcurrency` para no saturar la API
 - El contador de documentos se decrementa en N
 - Las variables de plantilla de email se reemplazan correctamente (`replaceMailVariables`)
 - **Prioridad:** Alta
 - **Nota:** Cloud Function `batchdocuments` → `apps/OpenSignServer/cloud/parsefunction/createBatchDocs.js`. Pool de workers async para concurrencia.
-

TC-DOC-10 — Rechazo de firma fuera de orden estricto (OPERATION_FORBIDDEN)

- **Precondición:**

- Un documento con `SendinOrder=true` y `SendInOrderStrict=true` fue enviado a Firmante A, B, C (en ese orden)
- Firmante A y B aún no han firmado
- El documento está en `contracts_Document` con `AuditTrail` vacío o solo con actividades de navegación (no "Signed")

- **Pasos:**

1. Firmante C (el tercero en orden) accede al link de firma
2. Completa la ceremonia de OTP/identidad
3. Intenta firmar el documento sin esperar a A y B
4. El frontend ejecuta la Cloud Function `signPdf` con `docId`, `reqUserId` (C), y firma
5. El backend valida en `PDF.js` (líneas 54-64) que `SendinOrder` y `SendInOrderStrict` están activados
6. El backend calcula el índice de C en el array `Placeholders`
7. El backend verifica el `AuditTrail` para encontrar si algún firmante previo a C (A o B) ya firmó
8. Como no hay entrada "Signed" de A en `AuditTrail`, se ejecuta `findPendingPriorSigner`

- **Resultado esperado:**

- La Cloud Function `signPdf` lanza una excepción `Parse.Error.OPERATION_FORBIDDEN`
- El mensaje de error es: "Strict signing order is enabled – please wait for the previous signers to complete their action before signing."
- El PDF no se firma
- El `AuditTrail` no se actualiza
- El frontend muestra el error al usuario

- **Prioridad:** Alta

- **Nota:**

- Verificado en `PDF.js` líneas 54-64:

```
if (reqUserId && _resDoc?.SendinOrder === true && _resDoc?.SendInOrderStrict === true) {
  const placeholders = _resDoc.Placeholders.filter(p => p?.Role !== 'prefill');
  const myIdx = findPlaceholderIndex(placeholders, reqUserId);
  if (myIdx > 0) {
    const pendingId = findPendingPriorSigner(placeholders, myIdx, _resDoc?.AuditTrail);
    if (pendingId) {
      throw new Parse.Error(Parse.Error.OPERATION_FORBIDDEN,
        'Strict signing order is enabled – please wait for the previous signers to complete their action before signing.'
      );
    }
  }
}
```

- Este es el caso de regresión real verificado en el código.

Ceremonia de firma

TC-DOC-11 — Firmante invitado nuevo (sin cuenta previa)

- **Precondición:** Un documento fue enviado. El email de firma es para un usuario que nunca antes interactuó con FibexSign.
 - **Pasos:**
 1. Firmante abre el link de firma (forma `login/<base64>` o `recipientSignPdf/:docId/:contactId`)
 2. Llega a `GuestLogin.jsx`
 3. Sistema detecta que el email es nuevo (no está en `contracts_Contactbook`)
 4. Se muestra formulario de datos básicos (`handleUserData`)
 5. Firmante rellena: nombre, email, teléfono (opcional), empresa (opcional), puesto (opcional)
 6. Hace clic en continuar/siguiente
 7. Se ejecuta Cloud Function `linkcontacttodoc`
(`apps/OpenSignServer/cloud/parsefunction/linkContactToDoc.js`)
 8. Se crea un contacto en `contracts_Contactbook` con `UserRole: 'contracts_Guest'`
 9. Se solicita OTP vía `SendOTPMailV1`
 - **Resultado esperado:**
 - El contacto se crea en `contracts_Contactbook` con `UserRole = 'contracts_Guest'`
 - El contacto se vincula al documento (aparece en `Signers` y `Placeholders`)
 - El ACL se actualiza para permitir al nuevo contacto leer/escribir
 - Se envía un OTP por email
 - Firmante puede continuar a la verificación OTP
 - **Prioridad:** Alta
 - **Nota:** Cloud Function `linkcontacttodoc` , verificado contra `GuestLogin.jsx` línea ~85.
-

TC-DOC-12 — Firmante invitado ya conocido

- **Precondición:**
 - Un documento fue enviado a un email ya existente en `contracts_Contactbook` del mismo usuario/tenant
 - El contacto ya tiene `UserRole = 'contracts_Guest'` de una firma anterior
 - **Pasos:**
 1. Firmante abre el link de firma
 2. Llega a `GuestLogin.jsx`
 3. Sistema detecta que el email ya existe en `contracts_Contactbook`
 4. Se salta el formulario de datos básicos
 5. Se muestra directamente un campo para confirmar email y solicitar OTP
 6. Firmante hace clic en "Enviar OTP" o similar
 - **Resultado esperado:**
 - No se crea un nuevo contacto
 - Se usa el contacto existente
 - Se envía OTP por email sin crear duplicados
 - El workflow continúa a verificación OTP
 - **Prioridad:** Media
 - **Nota:** Lógica en `GuestLogin.jsx` línea ~40 (búsqueda en `contracts_Contactbook` por email).
-

TC-DOC-13 — OTP es por documento, no global (OTP de documento A no sirve para documento B)

- **Precondición:**

- Dos documentos diferentes (A y B) fueron enviados al mismo email
- Ambos tienen `IsEnableOTP = true`
- Firmante ya solicitó y recibió OTPs para ambos

- **Pasos:**

1. Firmante abre el email de OTP para Documento A
2. Copia el código OTP de la línea "Este OTP es válido solo para el Documento A"
3. Abre el link de firma para Documento B
4. Sistema pide OTP para Documento B (porque `IsEnableOTP = true`)
5. Firmante intenta ingresar el OTP de Documento A en el campo de Documento B
6. Hace clic en "Verificar" o "Continuar"
7. El backend valida el OTP contra Documento B usando `AuthLoginAsMail`

- **Resultado esperado:**

- El OTP de Documento A es RECHAZADO para Documento B
- Se muestra un mensaje de error (ej. "Código OTP inválido" o "OTP expirado para este documento")
- Firmante debe solicitar el OTP correcto de Documento B
- El OTP correcto de Documento B funciona

- **Prioridad:** Alta

- **Nota:** Verificado en `signing-ceremony.md` línea ~14: "El OTP es por documento, no global". Cada `SendOTPMailV1` es por `docId`. Validación en `AuthLoginAsMail` (ver `authentication-and-multitenancy.md` para detalles).
-

TC-DOC-14 — Generación de firma por trazo (Draw)

- **Precondición:** Firmante pasó OTP, está en el panel de firma, viendo la pestaña "Draw" (trazo a mano alzada).
 - **Pasos:**
 1. Abrir pestaña "Draw" (`Draw.jsx`)
 2. Usar el ratón o pantalla táctil para dibujar una firma/rúbrica en el canvas
 3. Levantar el dedo/botón del ratón al terminar (evento `onEnd`)
 4. Verificar que el trazo se convierte a `dataURL`
 5. Si el usuario tiene una firma prefill guardada, verificar que puede cargarse con `canvasRef.current.fromDataURL(base64)`
 6. Hacer clic en "Confirmar firma" o "Aplicar"
 - **Resultado esperado:**
 - El trazo se renderiza en el canvas con `react-signature-canvas`
 - Al soltar, se convierte a `dataURL` (PNG base64)
 - Se pasa a `handleSignatureChange`
 - La firma se embebe en los widgets de firma del documento
 - Si hay prefill, se carga automáticamente sin necesidad de redibujar
 - **Prioridad:** Alta
 - **Nota:** Implementación en `Draw.jsx` , líneas ~50-70. Usa `react-signature-canvas` y `toDataURL()` .
-

TC-DOC-15 — Generación de firma por imagen subida (UploadImage)

- **Precondición:** Firmante está en el panel de firma, viendo la pestaña "UploadImage".
 - **Pasos:**
 1. Abrir pestaña "UploadImage" (`UploadImage.jsx`)
 2. Hacer clic en el recuadro de firma para abrir el selector de archivo
 3. Seleccionar una imagen PNG o JPEG desde el sistema de archivos
 4. La imagen se renderiza como preview en el recuadro
 5. Hacer clic en "Confirmar" o "Aplicar"
 - **Resultado esperado:**
 - Se acepta solo imágenes PNG y JPEG (`image/png` , `image/jpeg`)
 - La imagen se carga y renderiza como preview
 - Se embebe como firma en los widgets del documento
 - Funciona tanto para firma regular como para sellos (`isStampOrImage`)
 - **Prioridad:** Media
 - **Nota:** Implementación en `UploadImage.jsx` . Input type="file" hidden disparado por click.
-

TC-DOC-16 — Generación de firma por defecto del perfil (DefaultSignature)

- **Precondición:**
 - Firmante accedió como usuario logueado o invitado
 - El usuario tiene una firma por defecto guardada en su perfil (via `/managesign`)
 - Está en el panel de firma, pestaña "DefaultSignature"
 - **Pasos:**
 1. Abrir pestaña "DefaultSignature" (`components/pdf/tab/DefaultSignature.jsx`)
 2. Ver la firma/inicial guardada en el perfil (`defaultSignImg` / `myInitial` del store Redux)
 3. Una alerta de confirmación se muestra ("¿Usar esta firma en todos los widgets?")
 4. Hacer clic en "Sí" o "Confirmar"
 5. El backend consulta `getdefaultsignature` vía Cloud Function `getSignature.js`
 6. Se aplica automáticamente a todos los widgets de firma/inicial del documento
 - **Resultado esperado:**
 - La firma por defecto se carga sin errores
 - Se aplica a TODOS los widgets de firma/inicial (no requiere colocar firma en cada uno)
 - Si el usuario no tiene firma por defecto, se muestra un mensaje indicando que debe configurar una en `/managesign`
 - **Prioridad:** Media
 - **Nota:**
 - Persistencia via Cloud Functions `savesignature` / `managesign` → `saveSignature.js` , `manageSign.js` en `contracts_Signature`
 - Lectura via `getdefaultsignature` → `getSignature.js`
 - Ambas funciones exigen que `userId === request.user?.id` (no puede escribir la firma de otro)
-

TC-DOC-17 — Finalización con certificado — verifica hash SHA-256, timestamp, IP

- **Precondición:**

- Un documento con múltiples firmantes está en progreso
- El último firmante (o todos si no hay orden estricto) acaba de firmar
- El documento ahora está `IsCompleted = true`

- **Pasos:**

1. Último firmante ejecuta `signPdf` Cloud Function con su firma
2. Backend valida identidad (OTP si aplica) y orden (orden estricto si aplica)
3. Backend agrega entrada a `AuditTrail: { UserPtr, SignedUrl: '', Activity: 'Signed', ipAddress }`
4. Backend calcula si `isCompleted` comparando `COMPLETION_ACTIVITIES` contra `Placeholders`
5. Como el documento está completo, backend firma el PDF criptográficamente con PKCS#12
6. Backend calcula `generateDocumentHash(signedDocs)` — hash SHA-256 del buffer PDF firmado
7. Backend persiste `DocumentHash` en `contracts_Document`
8. Frontend llama Cloud Function `generatecertificate` (solo si `IsCompleted` y sin `CertificateUrl` previo)
9. Backend ejecuta `GenerateCertificate.js` (con `pdf-lib`)
10. Certificado contiene:
 - Id y nombre del documento
 - **Hash SHA-256 del documento** (`DocumentHash`)
 - Organización, fecha de creación, fecha de completado
 - Cantidad de firmantes y datos del "document originator" (nombre, email, **IP de origen**)
 - Un bloque por cada entrada del `AuditTrail` :
 - Nombre, email
 - "Security level: Email, OTP Auth" (solo si `IsEnableOTP = true` para ese documento)
 - Fecha de visualización (`ViewedOn`)
 - **Fecha de firma** (`SignedOn`)
 - **IP de la firma** (`ipAddress`)
 - Imagen de firma embebida (o "n/a" si no hay firma gráfica)
11. Certificado es firmado digitalmente con PKCS#12
12. Backend guarda URL del certificado en `CertificateUrl`

- **Resultado esperado:**

- El documento final es un PDF válido firmado digitalmente (PKCS#7)
- `DocumentHash` contiene un hash SHA-256 verificable
- El certificado es un PDF descargable con:
 - Hash SHA-256 visible
 - Timestamps completos (creación, completado, visualización por cada firmante, firma por cada firmante)
 - IP del documento originator
 - IP de cada firmante
 - "Security level: Email, OTP Auth" si aplica
- El certificado es también un PDF firmado digitalmente
- Ambos PDFs (documento + certificado) pueden verificarse posteriormente con una herramienta de validación de firma PKCS#7

- **Prioridad:** Alta
 - **Nota:**
 - `generateDocumentHash` en `PDF.js` línea ~100
 - `GenerateCertificate.js` línea ~45-80 (estructura del certificado)
 - Firma digital PKCS#12 usando `@signpdf/signpdf` + `@signpdf/signer-p12`
 - Verificación posterior via `VerifyDocument.jsx` (lectura de firma PKCS#7 en navegador)
-

TC-DOC-18 — Rechazo de documento (Decline) con `IsEnableOTP=true`

- **Precondición:**
 - Un documento no completado (ni archivado) fue enviado con `IsEnableOTP = true`
 - Firmante ha verificado su OTP y está logueado con sesión Parse válida
 - El documento aún está en progreso (no todas las firmas completadas)
 - **Pasos:**
 1. Firmante abre el documento en `SignyourselfPdf.jsx` o `PdfRequestFiles.jsx`
 2. En lugar de firmar, busca el botón "Rechazar", "Decline" o "Rechazar documento"
 3. Hace clic en el botón
 4. Se abre un modal/formulario para ingresar el motivo del rechazo
 5. Firmante escribe un motivo (ej. "No puedo firmar este documento porque X")
 6. Hace clic en "Rechazar" o "Confirm Decline"
 7. Se ejecuta la Cloud Function `declinedoc`
(`apps/OpenSignServer/cloud/parsefunction/declinedocument.js`)
 8. Como `IsEnableOTP = true` , el backend exige `request.user` (sesión Parse válida)
 9. El backend verifica que quien llama es el creador (`CreatedBy`) o está en `Signers`
 - **Resultado esperado:**
 - Se setean los campos: `IsDeclined: true` , `DeclineReason` , `DeclineBy` (puntero al usuario)
 - Si quien rechaza NO es el creador, se envía email al creador con motivo y link de vuelta al documento
 - El documento no puede ser abierto nuevamente para firma (queda rechazado)
 - Llamada repetida al mismo documento retorna `'document already declined'` sin re-notificar
 - El certificado no se genera (documento no está `IsCompleted`)
 - **Prioridad:** Media
 - **Nota:** Cloud Function `declinedoc` → `declinedocument.js` , verificación de autorización en línea ~30-40. Idempotente (línea ~70).
-

TC-DOC-19 — Rechazo de documento (Decline) con `IsEnableOTP=false`

- **Precondición:**
 - Un documento no completado (ni archivado) fue enviado con `IsEnableOTP = false`
 - Firmante NO tiene sesión Parse (está como invitado sin loguearse)
 - El documento está en progreso
 - **Pasos:**
 1. Firmante abre el link de firma como invitado (sin pasar por OTP si `IsEnableOTP = false`)
 2. Busca el botón "Rechazar", "Decline" o "Rechazar documento"
 3. Hace clic
 4. Se abre un modal para ingresar motivo
 5. Escribe motivo y hace clic en "Rechazar"
 6. Se ejecuta Cloud Function `declinedoc` con parámetro `userId` (no con `request.user`)
 7. Backend valida que el `userId` corresponde a un firmante o al creador comparando contra `Signers`
 - **Resultado esperado:**
 - El documento se marca como rechazado (`IsDeclined: true`)
 - El backend procesa el rechazo sin requerir sesión Parse
 - Email se envía al creador con motivo
 - El documento no se puede reabrir para firma
 - Comportamiento idéntico al TC-DOC-18, salvo que no se exige sesión Parse
 - **Prioridad:** Media
 - **Nota:** Con `IsEnableOTP = false` , el backend confía en `request.params.userId` para identificar al firmante, validando aún así que está autorizado.
-

Plantillas

TC-DOC-20 — Crear plantilla desde documento

- **Precondición:** Un documento completado o en progreso está abierto.
 - **Pasos:**
 1. Desde el documento (o desde el reporte de documentos completados), buscar la acción "Guardar como plantilla" ("Save as template")
 2. Hacer clic
 3. Se abre un modal para nombre de plantilla (reutiliza `EditTemplate.jsx`)
 4. Ingresar nombre, descripción (opcional), notas (opcional)
 5. Hacer clic en "Guardar plantilla"
 6. Se ejecuta Cloud Function `saveAsTemplate`
(`apps/OpenSignServer/cloud/parsefunction/saveAsTemplate.js`)
 - **Resultado esperado:**
 - Se crea un nuevo registro `contracts_Template` con:
 - `Name` , `Description` , `Note`
 - `URL` (PDF almacenado)
 - `Type` `≠ "Folder"`
 - `IsArchive` `≠ true`
 - `Signers[]` , `Placeholders[]` , `SendinOrder` , `SendInOrderStrict` , `IsEnableOTP` (copiados del documento)
 - `CreatedBy` (usuario autenticado)
 - No hay `AuditTrail` (es una plantilla, no un documento enviado)
 - La plantilla aparece en el reporte de plantillas (`TemplatesReport.jsx`)
 - **Prioridad:** Alta
 - **Nota:** Cloud Function `saveAsTemplate` → `saveAsTemplate.js` . Usa `getPdfMetadataHash` para hash de estructura.
-

TC-DOC-21 — Usar plantilla para generar documento (Use)

- **Precondición:** Una plantilla existe en `contracts_Template`.
 - **Pasos:**
 1. Ir al reporte de plantillas (`/report/6TeaPr321t` o similar)
 2. Encontrar la plantilla
 3. Hacer clic en el botón "Usar" o "Use"
 4. Se abre un editor con el contenido de la plantilla (PDF, widgets, destinatarios)
 5. Puede editarse libremente (cambiar destinatarios, widgets, metadatos)
 6. Hacer clic en "Enviar" o "Guardar como borrador"
 - **Resultado esperado:**
 - Se crea un nuevo documento `contracts_Document` (basado en la plantilla)
 - El documento tiene los mismos widgets, destinatarios, flags que la plantilla
 - El documento NO afecta a la plantilla original
 - El documento puede modificarse independientemente
 - La plantilla sigue disponible para uso futuro
 - **Prioridad:** Alta
 - **Nota:** Acción en `TemplatesReport.jsx` botón "Use", dispara flujo de creación de documento con datos precargados de la plantilla.
-

TC-DOC-22 — Compartir plantilla con equipo (Share with team)

- **Precondición:** Una plantilla existe, creada por el usuario actual.
 - **Pasos:**
 1. En el reporte de plantillas, buscar la plantilla
 2. Hacer clic en "Opciones" (menú de 3 puntos) o similar
 3. Seleccionar "Compartir con equipo" o "Share with team"
 4. Se abre un modal para seleccionar usuarios/equipos con los que compartir
 5. Seleccionar uno o más usuarios o un equipo
 6. Hacer clic en "Compartir" o "Confirm"
 - **Resultado esperado:**
 - Los usuarios/equipos seleccionados pueden ver y usar la plantilla en su reporte de plantillas
 - El ACL de `contracts_Template` se actualiza para dar acceso de lectura/escritura a los usuarios compartidos
 - La plantilla es filtrada por `getReport.js` según filtros de equipos y usuarios compartidos (verificado en `reportsJson.js`)
 - **Prioridad:** Media
 - **Nota:** Acción en menú de `TemplatesReport.jsx`. Backend filtra templates en `getReport.js` por usuario autenticado + equipos + usuarios compartidos.
-

TC-DOC-23 — Cambios futuros a plantilla NO afectan documentos ya enviados (regresión)

- **Precondición:**

- Una plantilla original (A) existe con 2 destinatarios, 3 widgets
- Se utilizó la plantilla A para crear y enviar 2 documentos (D1, D2)
- Ambos documentos D1 y D2 están en progreso o completados

- **Pasos:**

1. Abrir la plantilla A para edición
2. Agregar un tercer destinatario a la plantilla
3. Agregar 2 widgets adicionales a la plantilla
4. Cambiar el flag `SendInOrderStrict` de false a true
5. Guardar cambios a la plantilla
6. Abrir el reporte de documentos
7. Verificar los documentos D1 y D2:
 - D1: verificar que sigue teniendo 2 destinatarios, 3 widgets, `SendInOrderStrict = false`
 - D2: verificar que sigue teniendo 2 destinatarios, 3 widgets, `SendInOrderStrict = false`
8. Abrir nuevamente la plantilla A
9. Crear un nuevo documento D3 desde la plantilla
10. Verificar que D3 tiene 3 destinatarios, 5 widgets, `SendInOrderStrict = true` (los cambios de la plantilla)

- **Resultado esperado:**

- D1 y D2 NO cambian (siguen con la configuración original)
- D3 tiene la nueva configuración de la plantilla actualizada
- La plantilla es una "foto" del documento en el momento de la creación, no un vínculo dinámico
- Los cambios posteriores a la plantilla solo afectan a documentos nuevos creados después

- **Prioridad:** Alta

- **Nota:**

- Verificado implícitamente en `docs/reports.md` línea ~97 (tour de plantillas menciona esto)
- Arquitectura: `contracts_Template` y `contracts_Document` son independientes tras la creación
- No hay vínculo back-reference que actualice documentos previos

Resumen de casos de prueba

Total de casos: 23

Sección	Casos	IDs
Preparación	7	TC-DOC-01 a TC-DOC-07
Envío	3	TC-DOC-08 a TC-DOC-10
Ceremonia de firma	7	TC-DOC-11 a TC-DOC-19
Plantillas	4	TC-DOC-20 a TC-DOC-23

Pendientes de verificación

1. **Copy exacto de botones/etiquetas UI:** Los casos usan descripciones funcionales (ej. "botón Rechazar") en lugar de copias literales. Verificar contra los componentes reales si es necesario.
 2. **Rutas exactas de URLs:** Los links de firma mencionan forma `login/<base64>` o `recipientSignPdf/:docId/:contactId` — confirmar contra el enrutador actual.
 3. **Mensajes de error exactos:** Algunos mensajes se citan del código (ej. OPERATION_FORBIDDEN), pero otros se describen por función. Verificar contra las strings reales si se requiere validación exacta.
 4. **Tiempos de OTP:** No especificados en los docs (expiration, reintento, etc.). Asumir valores por defecto de Parse.
 5. **Verificación de firma PKCS#7:** El doc menciona que existe `VerifyDocument.jsx` pero no entra en detalles ASN.1. Caso TC-DOC-17 cubre la generación, no la verificación posterior.
-

Generado: 2026-09-24

Fuente: `document-preparation-and-sending.md`, `signing-ceremony.md`, `reports.md`

QA — Casos de Prueba: Email Builder, OpenSignDrive, Reportes/Contactos, Internacionalización

Última actualización: 2026-09-24

Estado: Casos derivados de documentación técnica verificada (docs/email-builder.md , docs/opensign-drive.md , docs/reports.md , docs/internationalization.md)

Email Builder

TC-FEAT-01 — Editar plantilla visualmente y exportar HTML sin guardar directo

- **Precondición:** Usuario autenticado, accede a `/emailbuilder` o `/emailbuilder#sample/requestemail`
- **Pasos:**
 1. Abre `/emailbuilder` (o elige una plantilla de ejemplo vía `#sample/requestemail`)
 2. Arrastra bloques (Heading, Text, Button, Image) al panel central (TemplatePanel)
 3. Edita propiedades de un bloque (color, tamaño, contenido) usando el Inspector drawer derecho
 4. Visualiza la plantilla en vista previa (tab "preview") y en desktop/mobile
 5. Haz clic en "ShareButton" (copia HTML al portapapeles) o "DownloadJson" (descarga archivo)
 6. Verifica que NO existe un botón "Guardar" o "Guardar plantilla" en el EmailBuilder
 7. Navega a Preferences > Email (`apps/OpenSign/src/components/preferences/tabs/Email.jsx`)
 8. Pega el HTML exportado en el campo de EmailBodyEditor
 9. Haz clic en "Guardar plantilla de solicitud" (o completitud según corresponda)
- **Resultado esperado:**
 - El HTML se renderiza correctamente en la vista previa
 - No hay botón de guardar directo en EmailBuilder
 - El HTML copiado/descargado se pega exitosamente en Preferences > Email
 - La plantilla se persiste en `contracts_Users.RequestBody` (o `CompletionBody`) tras hacer clic en guardar en Preferences
- **Prioridad:** Alta
- **Nota:** Este es un caso de regresión arquitectónica. El flujo esperado es: EditBuilder → Copiar/Descargar HTML → Pegarlo en Preferences. Si en una versión futura aparece un botón de guardar directo en el EmailBuilder, este test debe fallar y alertar sobre un cambio arquitectónico no documentado.

TC-FEAT-02 — Variables de reemplazo en email de solicitud enviado

- **Precondición:** Usuario tiene una plantilla personalizada guardada en `Preferences > Email` que incluye variables: `{{sender_name}}`, `{{receiver_name}}`, `{{document_title}}`, `{{signing_url}}`
 - **Pasos:**
 1. Crea un documento nuevo (plantilla predefinida o personalizada)
 2. Agrega un firmante con nombre conocido (ej: "Juan Pérez")
 3. Establece un título de documento (ej: "Contrato ABC")
 4. Envía el documento para firma
 5. Verifica el email recibido por el firmante
 6. Abre el email y revisa el contenido del cuerpo (body)
 - **Resultado esperado:**
 - `{{sender_name}}` se reemplaza con el nombre del usuario que envía
 - `{{receiver_name}}` se reemplaza con "Juan Pérez"
 - `{{document_title}}` se reemplaza con "Contrato ABC"
 - `{{signing_url}}` se reemplaza con un link válido de firma (generado en `createBatchDocs.js`)
 - No aparecen variables sin reemplazar (ej: literal `{{receiver_name}}`)
 - **Prioridad:** Alta
 - **Nota:** Las variables se reemplazan en el backend (`apps/OpenSignServer/cloud/parsefunction/createBatchDocs.js` para email de solicitud, `pdf/PDF.js` para email de completitud) vía `sendmailv3`. Si una variable falta en la plantilla, la verificación debe confirmar que no aparece ningún error en logs ni en el email.
-

OpenSignDrive

TC-FEAT-03 — Navegación de carpetas, subida de archivo y búsqueda

- **Precondición:** Usuario autenticado, accede a `/opensigndrive`
- **Pasos:**
 1. Verifica que el breadcrumb muestra "OpenSignDrive" (raíz)
 2. Hace clic en "+ Nuevo" y selecciona "Crear carpeta"
 3. Ingresa un nombre único para la carpeta (ej: "Contratos Q4")
 4. Verifica que la carpeta aparece en la lista (grid o tabla)
 5. Hace clic en la carpeta para navegar dentro
 6. Verifica que el breadcrumb se actualiza: "OpenSignDrive > Contratos Q4"
 7. En el drive, busca un documento existente usando el campo de búsqueda (debounce 300ms)
 8. Verifica que los resultados coinciden con el término (búsqueda case-insensitive)
- **Resultado esperado:**
 - Carpeta se crea exitosamente y aparece en la lista
 - Navegación por breadcrumb funciona
 - La búsqueda retorna documentos cuyo nombre coincide (caso insensible)
 - El refinado de búsqueda y el cambio de vista (grid/tabla) son fluidos
- **Prioridad:** Alta
- **Nota:** La búsqueda invoca `filterdocs` con debounce 300ms. La paginación usa infinite scroll con `IntersectionObserver`.

TC-FEAT-04 — Renombrar/mover/eliminar con validación de nombre único (regresión de duplicado)

- **Precondición:** Usuario autenticado, dos navegadores abiertos en `/opensigndrive`, mismo usuario en ambos
- **Pasos:**
 1. **Navegador A:** En la carpeta raíz, haz clic en "+ Nuevo" → "Crear carpeta", ingresa nombre "Test Folder"
 2. **Navegador B:** Simultáneamente (dentro de 2 segundos), haz clic en "+ Nuevo" → "Crear carpeta", intenta ingresar el mismo nombre "Test Folder"
 3. Verifica si el sistema permite crear dos carpetas con el mismo nombre en el mismo nivel (regresión esperada según doc: NO hay validación server-side, solo client-side)
 4. Si ambas se crean, documenta el comportamiento (bug de duplicado)
 5. En caso de que una falle, verifica el mensaje de error mostrado al usuario
- **Resultado esperado:**
 - Idealmente: Una de las dos requests falla con mensaje "Carpeta ya existe en este nivel"
 - Observado (según docs): Es posible que ambas se creen si las requests concurren, porque la validación es client-side (`Parse.Query` en el cliente) y no hay índice único DB-side
- **Prioridad:** Media (es una regresión conocida no corregida)
- **Nota:** La línea 296–302 de `Opensigndrive.jsx` realiza la validación `Parse.Query("contracts_Document").equalTo("Name", ...)` antes de crear, pero esto es vulnerable a race conditions si dos usuarios/tabs envían la request a la vez. El documento señala que "no hay validación server-side de nombre único — la validación ocurre client-side". Este test verifica la existencia del bug.

TC-FEAT-05 — Verificar soft delete (IsArchive: true) y no borrado físico

- **Precondición:** Usuario autenticado, accede a `/opensigndrive`, hay documentos/carpetas existentes
- **Pasos:**
 1. Haz clic derecho en un documento o carpeta (o usa el menú contextual)
 2. Selecciona "Eliminar" / "Delete"
 3. Confirma la eliminación
 4. Verifica que el documento desaparece de la lista en la UI
 5. Abre la consola del navegador (DevTools → Network)
 6. Ejecuta un `Parse.Query` manual contra `contracts_Document` incluyendo archivados:

```
const query = new Parse.Query("contracts_Document");
query.include("IsArchive");
const result = await query.find();
// Busca el documento eliminado, verifica IsArchive: true
```

7. (Alternativa si no puedes acceder a console): Intenta acceder al documento vía URL directa o API si el endpoint lo expone
- **Resultado esperado:**
 - El documento desaparece de la lista en la UI (no se renderiza porque `getDrive` filtra `IsArchive !== true`)
 - El registro en `contracts_Document` persiste con `IsArchive: true` (no se borra físico)
 - Consultas futuras con filtro `IsArchive !== true` no lo incluyen
 - **Prioridad:** Alta (auditoría, compliance)
 - **Nota:** Línea 158 de `Opensigndrive.jsx` ejecuta `PUT /classes/contracts_Document/:docId` con `{ IsArchive: true }`. No hay borrado físico, solo soft delete.

Reportes y Contactos

TC-FEAT-06 — Documento en estado Draft aparece en reporte "Draft Documents"

- **Precondición:** Usuario autenticado, ha creado al menos un documento sin firmar (draft, sin destinatarios ni placeholders iniciales)
- **Pasos:**
 1. Accede a `/report/ByHuevtCFY` (ID del reporte "Draft Documents")
 2. Verifica que el documento Draft aparece en la lista
 3. Haz clic en el documento para navegarlo
 4. Verifica que puedes editarlo (estado = Draft)
- **Resultado esperado:**
 - El documento Draft se lista correctamente bajo "Draft Documents"
 - Acciones disponibles: Edit, Delete
 - No aparece en otros reportes de estado (Completed, Declined, etc.)
- **Prioridad:** Alta
- **Nota:** El reporte Draft se filtra por `SignedUrl` ausente y otros campos según `getStatusBadgeConfig()` (línea 301 de `Opensigndrive.jsx`).

TC-FEAT-07 — Reportes de documentos por estado (completo)

- **Precondición:** Usuario tiene documentos en variados estados (draft, pendiente de firma, en progreso, completado, rechazado, expirado)
- **Pasos:**
 1. Accede a cada uno de estos reportes:
 - `/report/ByHuevtCFY` (Draft)
 - `/report/4Hhwbp482K` (Need your sign — esperando mi firma)
 - `/report/1MwEuxLEkF` (In-progress)
 - `/report/kQUoW4hUXz` (Completed)
 - `/report/UPr2Fm5WY3` (Declined)
 - `/report/zNqBHXHsYH` (Expired)
 2. Para cada reporte, verifica que los documentos listados tienen el estado esperado (revisa el badge de estado en la UI o los campos de base de datos)
 3. Verifica que cada documento aparece en exactamente UN reporte (no duplicado en múltiples estados)
- **Resultado esperado:**
 - Cada documento se clasifica correctamente según su estado
 - Draft: sin `SendedUrl`, sin destinatarios
 - In-progress: con `SignedUrl` + `Signers` pendientes
 - Completed: `IsCompleted` = true
 - Declined: `IsDeclined` = true
 - Expired: fecha expiración < ahora
 - Ningún documento duplicado en reportes
- **Prioridad:** Alta
- **Nota:** La lógica de clasificación está en `getStatusBadgeConfig()` y en el backend `reportsJson.js`. El orden de evaluación es: `IsCompleted` > `IsDeclined` > `SignedUrl` ausente > Expirado > In Progress.

TC-FEAT-08 — Importación de contactos - CSV válido y límite de 100 registros

- **Precondición:** Usuario autenticado, accede a Reportes → Contactbook → botón de importar (o `/report/contacts`)

- **Pasos:**

1. Prepara un archivo CSV válido con 50 registros:

- Headers: `Name` , `Email` , `Phone` (opcional), `Company` (opcional), `JobTitle` (opcional)
- Ejemplo:

```
Name,Email,Phone,Company
John Doe,john@example.com,555-1234,Acme Corp
Jane Smith,jane@example.com,555-5678,Beta Inc
...
```

2. Haz clic en "Importar contactos" / "Import Contacts"

3. Selecciona el archivo CSV

4. Verifica que la importación completa exitosamente

5. Revisa que los contactos aparecen en el Contactbook (columnas: Name, Email, Phone, Company, JobTitle)

6. Repite con 100 registros (máximo permitido)

7. Intenta importar un archivo con 101 registros

- **Resultado esperado:**

- CSV válido con ≤ 100 registros se importa exitosamente
- Contactos aparecen en el Contactbook
- CSV con 101+ registros: (pendiente de verificar si genera error en la UI o se trunca silenciosamente)

- **Prioridad:** Media

- **Nota:** La validación del máximo se menciona en `docs/reports.md` línea 75 ("máx. 100 registros"). El componente es `ImportContact.jsx` , Cloud Function es `createbatchcontact` .

TC-FEAT-09 — Importación de contactos - emails inválidos y duplicados

- **Precondición:** Usuario en Contactbook, acceso a formulario de importación
- **Pasos:**
 1. Crea un CSV con emails inválidos:

```
Name,Email
Bad Email 1,notanemail
Bad Email 2,@example.com
Good Email,valid@example.com
```

2. Intenta importar
3. Verifica si la importación rechaza todo el lote o solo las filas con emails inválidos
4. Crea un CSV con contactos duplicados (mismo email):

```
Name,Email
John,john@example.com
John Copy,john@example.com
```

5. Intenta importar
 6. Verifica si se detectan duplicados (en el mismo archivo o contra contactos existentes)
- **Resultado esperado:**
 - Emails inválidos: Se rechaza la fila o se muestra un error de validación (regex de email)
 - Duplicados en el mismo archivo: Se importa solo un registro o se rechaza el lote
 - Duplicados vs. contactos existentes: (pendiente de verificar el comportamiento — ¿sobrescribe o rechaza?)
 - **Prioridad:** Media
 - **Nota:** La validación de email usa regex (línea 78 de `docs/reports.md`). No hay documentación clara sobre el manejo de duplicados.
-

Internacionalización

TC-FEAT-10 — Cambiar idioma y persistencia en localStorage

- **Precondición:** Usuario autenticado, accede a cualquier página del app (ej: Dashboard o Reports)
- **Pasos:**
 1. Ubica el selector de idioma (componente `SelectLanguage.jsx` , típicamente en navegación/settings)
 2. Verifica que los 7 idiomas están disponibles: English (en), Español (es), Français (fr), Italiano (it), Deutsch (de), हिन्दी (hi), 한국어 (kr)
 3. Selecciona un idioma diferente al actual (ej: de "en" a "es")
 4. Verifica que la UI se traduce (textos, labels, botones cambian de idioma)
 5. Recarga la página (F5 o Ctrl+R)
 6. Verifica que el idioma seleccionado persiste (no vuelve a inglés)
 7. Abre la consola del navegador (DevTools → Console)
 8. Ejecuta `localStorage.getItem('i18nextLng')` y verifica que devuelve el código del idioma seleccionado (ej: "es")
- **Resultado esperado:**
 - Los 7 idiomas están disponibles en el selector
 - El cambio de idioma es instantáneo
 - La UI se traduce correctamente
 - Tras recargar, el idioma persiste
 - `localStorage.i18nextLng` contiene el código del idioma
- **Prioridad:** Alta
- **Nota:** La detección de idioma tiene prioridad: `localStorage` > `navigator` > fallback "en". La carga de archivos es dinámica vía HTTP Backend (`/locales/{{lng}}/translation.json`).

TC-FEAT-11 — Regresión: Desincronización de clave `attachments-docx-size-limit` entre EN y ES

- **Precondición:** Usuario en español (es), accede a una sección que muestra texto sobre límite de tamaño de adjuntos DOCX
 - **Pasos:**
 1. Cambia idioma a Inglés (en)
 2. Navega a una sección que mencione "docx size limit" o un componente que muestre esta restricción (ej: formulario de upload, página de validación de documento)
 3. Verifica que aparece texto traducido (ej: "Attachments DOCX size limit: 10 MB")
 4. Cambia idioma a Español (es)
 5. Navega a la misma sección
 6. Observa qué texto aparece
 - **Resultado esperado:**
 - EN: Texto traducido correctamente (clave `attachments-docx-size-limit` existe en `en/translation.json`)
 - ES: Idealmente, texto traducido. Observado (según doc): Es posible que muestre la clave literal "attachments-docx-size-limit" en lugar de texto traducido, indicando que la clave falta en `es/translation.json`
 - **Prioridad:** Media (regresión de UX menor)
 - **Nota:** Línea 207–209 de `docs/internationalization.md` señala: "Hallazgo verificado — Comparando `translation.json` entre `en` y `es`, existe una discrepancia: `es` tiene `attachments-docx-upload-limit` pero falta `attachments-docx-size-limit`." Este test verifica la manifestación del bug en la UI. Requiere reproducir localizando el componente exacto que usa esta clave (pendiente de ubicar en el código).
-

Resumen

Total de casos de prueba escritos: 11 (TC-FEAT-01 a TC-FEAT-11)

Desglose por sección:

- Email Builder: 2 casos
- OpenSignDrive: 3 casos
- Reportes y Contactos: 4 casos
- Internacionalización: 2 casos

Observaciones sobre verificabilidad:

1. **Email Builder (TC-01, TC-02):** Totalmente verificable contra documentación. Variables de reemplazo confirmadas en `createBatchDocs.js` y `pdf/PDF.js`.
2. **OpenSignDrive (TC-03, TC-04, TC-05):** Verificables. TC-04 documenta una regresión conocida (validación client-side sin índice unique server-side). TC-05 verificable mediante API o Parse SDK.
3. **Reportes (TC-06, TC-07):** Reportes verificables en `reportsJson.js`. Lógica de estado en `getStatusBadgeConfig()`. TC-07 requiere documentos en distintos estados para reproducir.
4. **Contactos (TC-08, TC-09):** Máximo de 100 registros confirmado en doc. Validación de email por regex confirmada. Comportamiento de duplicados NO está documentado → marcado como "(pendiente de verificar)".
5. **Internacionalización (TC-10, TC-11):** TC-10 completamente verificable (7 idiomas confirmados, localStorage y fallback en doc). TC-11 es una regresión verificada en el documento → síntoma exacto (clave faltante en `es`) confirmado en línea 207–209.

Items pendiente de verificar:

- TC-FEAT-04: Exacto comportamiento con race condition (¿error o duplicado creado?)
- TC-FEAT-09: Cómo se manejan duplicados de contactos (¿sobrescribe o rechaza?)
- TC-FEAT-08/09: Ubicación exacta del botón "Importar" en la UI de Contactbook
- TC-FEAT-11: Componente exacto que muestra la clave `attachments-docx-size-limit` (necesita búsqueda en código)

Casos de seguridad conocidos (sin corregir)

Estos casos verifican comportamiento **ya documentado como hallazgo de seguridad** en `docs/cloud-functions-catalog.md`, encontrado durante la redacción de la documentación técnica y **no corregido** (fuera de alcance del issue de documentación). Están acá para que QA los tenga trackeados como regresión pendiente, no para "descubrirlos" de nuevo — si en algún momento se corrigen, estos casos deberían empezar a fallar en su forma actual (`request.user` ausente ya no debería alcanzar) y hay que actualizarlos.

TC-SEC-01 — `getcontact` sin chequeo de `request.user`

- **Precondición:** Dos usuarios de tenants distintos (A y B), cada uno con al menos un contacto propio en `contracts_Contactbook.contactId` de un contacto de B conocido/obtenible.
- **Pasos:**
 1. Autenticarse como usuario del tenant A.
 2. Invocar `Parse.Cloud.run('getcontact', { contactId: '<id de un contacto del tenant B>' })` (vía consola de red del navegador o un cliente HTTP directo contra el endpoint Cloud Function).
- **Resultado esperado (estado actual, hallazgo sin corregir):** la función devuelve el contacto de B — `cloud/parsefunction/getContact.js` no valida `request.user` contra el dueño del contacto, usa `useMasterKey`.
- **Prioridad:** Alta
- **Nota:** Este es el resultado *actual* documentado, no el deseado — trackear como hallazgo de seguridad pendiente, referenciado en `docs/cloud-functions-catalog.md` línea ~267-272.

TC-SEC-02 — `updatecontacttour` sin chequeo de `request.user`

- **Precondición:** Igual que TC-SEC-01, con un `contactId` de otro tenant.
- **Pasos:**
 1. Autenticarse como usuario del tenant A.
 2. Invocar `Parse.Cloud.run('updatecontacttour', { contactId: '<id de contacto ajeno>' })`.
- **Resultado esperado (estado actual, hallazgo sin corregir):** la función marca `TourStatus.requestSign: true` sobre un contacto que no pertenece al caller — sin chequeo explícito de `request.user`, `useMasterKey`.
- **Prioridad:** Media
- **Nota:** Impacto menor que TC-SEC-01/03 (solo modifica un flag de UI/tour), pero mismo patrón de falta de autorización — `docs/cloud-functions-catalog.md` línea ~274-279.

TC-SEC-03 — `linkcontacttodoc` autoriza solo por coincidencia email/placeholder

- **Precondición:** Un documento con al menos un `Placeholder` con un email de firmante conocido.
- **Pasos:**
 1. Sin sesión autenticada (o con sesión de un usuario ajeno al documento), invocar `Parse.Cloud.run('linkcontacttodoc', { email: '<email de un placeholder del documento>', docId, name, phone, jobTitle, company })`.
 2. Repetir con un `email` que NO está entre los `Placeholders` del documento.
- **Resultado esperado:** paso 1 debería otorgar ACL de lectura/escritura sobre el documento al contacto (comportamiento esperado del flujo de firmante invitado); paso 2 debería fallar con `OPERATION_FORBIDDEN / 'unauthorized'`.
- **Prioridad:** Alta
- **Nota:** No es un hallazgo nuevo — es el mecanismo de diseño del flujo de firmante invitado (autoriza por posesión del email correcto, no por sesión). El caso de prueba es para confirmar que el segundo camino (email fuera de los `Placeholders`) sigue bloqueado — `docs/cloud-functions-catalog.md` línea ~135-140.

TC-SEC-04 — `updateemailtemplates` no valida que `tenantId` pertenezca al caller

- **Precondición:** Dos usuarios de tenants distintos (A y B). Usuario A conoce o puede adivinar/enumerar el `tenantId` de B.
- **Pasos:**
 1. Autenticarse como usuario del tenant A.
 2. Invocar `Parse.Cloud.run('updateemailtemplates', { tenantId: '<tenantId de B>', details: { RequestBody: '<HTML de prueba>' } })`.
 3. Verificar como usuario B si sus plantillas de email fueron modificadas.
- **Resultado esperado (estado actual, hallazgo sin corregir):** a diferencia de `updatetenant`, esta función **no valida** que el `tenantId` recibido coincida con el tenant del caller — solo lo usa para el `include` de la query. El registro de `contracts_Users` del caller termina actualizado igual (el catálogo aclara que los templates viven en el registro de usuario extendido, no en `partners_Tenant`, pese al nombre del parámetro) — verificar el resultado real contra ese detalle antes de marcar el caso como bloqueante puro de cross-tenant.
- **Prioridad:** Alta
- **Nota:** Verificar con precisión el efecto real (¿modifica el propio registro del caller pese al `tenantId` ajeno, o el de B?) — el catálogo señala la falta de validación pero el efecto exacto de cross-tenant queda para confirmar en este caso de prueba. `docs/cloud-functions-catalog.md` línea ~367-372.

TC-SEC-05 — `POST /decryptpdf` sin autenticación

- **Precondición:** Un PDF protegido con contraseña conocida.
- **Pasos:**
 1. Sin ningún header de sesión ni master key, hacer `POST` directo al endpoint `/decryptpdf` con el archivo multipart (`file`) y `body.password` correcto.
 2. Repetir con `password` incorrecta.
- **Resultado esperado (estado actual, hallazgo sin corregir):** paso 1 devuelve `200` con el PDF descifrado en binario — el endpoint **no verifica sesión ni master key**, cualquiera que lo alcance puede intentar descifrar un PDF subido; paso 2 devuelve `401 { error: 'Incorrect password.' }`.
- **Prioridad:** Alta
- **Nota:** Es el hallazgo más directo de los 5 — no requiere ninguna sesión previa, solo alcanzar el endpoint. `docs/cloud-functions-catalog.md` línea ~399-404.